

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №3
Вариант №3802

Группа Р3208

Студент Горин Семён Дмитриевич

Преподаватель Наумова Надежда Александровна

Содержание

1. Текст задания	3
2. Листинг билд-файла	3
3. Пример исполнения цели alt	11
4. Листинг тестов	11
5. Вывод	13

1. Текст задания

Внимание! У разных вариантов разный текст задания!

Написать сценарий для утилиты [Apache Ant](#), реализующий компиляцию, тестирование и упаковку в jar-архив кода проекта из [лабораторной работы №3](#) по дисциплине "Веб-программирование".

Каждый этап должен быть выделен в отдельный блок сценария; все переменные и константы, используемые в сценарии, должны быть вынесены в отдельный файл параметров; MANIFEST.MF должен содержать информацию о версии и о запуске класса.

Сценарий должен реализовывать следующие цели (targets):

1. **compile** -- компиляция исходных кодов проекта.
2. **build** -- компиляция исходных кодов проекта и их упаковка в исполняемый jar-архив. Компиляцию исходных кодов реализовать посредством вызова цели **compile**.
3. **clean** -- удаление скомпилированных классов проекта и всех временных файлов (если они есть).
4. **test** -- запуск junit-тестов проекта. Перед запуском тестов необходимо осуществить сборку проекта (цель **build**).
5. **music** - воспроизведение музыки по завершению сборки (цель **build**).
6. **alt** - создаёт альтернативную версию программы с измененными именами переменных и классов (используя задание replace/replaceregexp в файлах параметров) и упаковывает её в jar-архив. Для создания jar-архива использует цель **build**.

Вопросы к защите лабораторной работы:

1. Тестирование ПО. Цель тестирования, виды тестирования.
2. Модульное тестирование, основные принципы и используемые подходы.
3. Пакет JUnit, основные API.
4. Системы автоматической сборки. Назначение, принципы работы, примеры систем.
5. Утилита make. Make-файлы, цели и правила.
6. Утилита Ant. Сценарии сборки, цели и команды.

- 1) Используем JUnit 5
- 2) Сценарии сборки необходимо реализовать на gradle. При этом, использовать встроенные задачи gradle (например, реализовать свой build через команду build из коробки и т.д.) запрещено.
- 3) Тестовые сценарии необходимо реализовать для компонента, проверяющего попадание в область

2. Листинг билд-файла

```
1: buildscript {
2:     repositories { mavenCentral() }
3:     dependencies {
4:         classpath 'com.github.javaparser:javaparser-core:3.28.1'
5:     }
6: }
7:
8: plugins {
9:     id 'java'
10:    id 'war'
11:    id('io.ebean') version '14.1.0'
12: }
13:
14:
15: repositories {
16:     mavenCentral()
17: }
18:
19: java {
20:     sourceCompatibility = JavaVersion.VERSION_17
21:     targetCompatibility = JavaVersion.VERSION_17
22: }
23:
24: ext {
25:     junitVersion = '5.10.2'
26:     mockitoVersion = '5.23.0'
27: }
28:
```

```

29: base {
30:     archivesName = 'ROOT'
31: }
32:
33:
34: tasks.withType(JavaCompile) {
35:     options.encoding = 'UTF-8'
36: }
37:
38: dependencies {
39:     implementation('io.ebean:ebean-postgres:14.1.0')
40:     compileOnly('jakarta.platform:jakarta.jakartaee-api:11.0.0')
41:     compileOnly("org.projectlombok:lombok:1.18.42")
42:
43:     annotationProcessor("org.projectlombok:lombok:1.18.42")
44:     annotationProcessor('io.ebean:querybean-generator:14.1.0')
45:
46:     implementation("de.mkammerer:argon2-jvm:2.11")
47:     implementation("org.eclipse.microprofile.jwt:microprofile-jwt-
auth-api:2.1")
48:     implementation("jakarta.json:jakarta.json-api:2.1.2")
49:     implementation("org.postgresql:postgresql:42.7.3")
50:     runtimeOnly("org.eclipse.parsson:parsson:1.1.5")
51:     implementation("io.ebean:ebean-ddl-generator:17.1.0")
52:
53:
54:     implementation("org.jboss.weld.se:weld-se-core:3.0.6.Final")
55:     implementation("org.jboss.weld.module:weld-web:3.0.6.Final")
56:
57:     testImplementation("org.mockito:mockito-core:${mockitoVersion}")
58:     testImplementation("org.mockito:mockito-junit-jupiter:
${mockitoVersion}")
59:
60:     testImplementation("org.jboss.weld:weld-junit5:5.0.3.Final")
61:
62:     testCompileOnly("org.projectlombok:lombok:1.18.42")
63:     testAnnotationProcessor("org.projectlombok:lombok:1.18.42")
64:     testImplementation("org.junit.jupiter:junit-jupiter-api:
${junitVersion}")
65:     testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:
${junitVersion}")
66:     testRuntimeOnly("org.junit.platform:junit-platform-launcher")
67: }
68:
69: tasks.register('myCompile', MyJavaCompile) {
70:     source.from fileTree('src/main/java')
71:     processorClasspath.from configurations.annotationProcessor
72:     compileClasspath.from configurations.compileClasspath
73:     destinationDirectory = layout.buildDirectory.dir('my-classes')
74:     resources.from fileTree('src/main/resources')
75: }
76:
77: tasks.register('buildWar', MyBuildWar) {

```

```

78:     sourceDirectory = layout.buildDirectory.dir('my-classes')
79:     baseName = base.archivesName.get()
80:     runtimeClasspath.from configurations.runtimeClasspath
81:     outputFile = layout.buildDirectory.file('libs/' +
    base.archivesName.get() + '.war')
82: }
83: tasks.register('myClean', Delete) {
84:     delete layout.buildDirectory
85: }
86:
87: tasks.register('myTest', Test) {
88:     dependsOn myBuild
89:     useJUnitPlatform()
90:     jvmArgs "-javaagent:${configurations.testRuntimeClasspath.find
    { it.name.contains('mockito-core') }}"
91: }
92:
93: tasks.register('music') {
94:     dependsOn myBuild
95:     doLast {
96:         def process = ['ffplay', '-nodisp', 'https://pool.anison.fm/
    AniSonFM(320)?nocache=0.7628170478546088'].execute()
97:
98:         sleep(1000 * 20)
99:         process.destroy()
100:     }
101: }
102:
103: tasks.register('alt', MyJavaCompile) {
104:     dependsOn uwuifySources
105:     source.from fileTree(layout.buildDirectory.dir('uwu-src'))
106:     processorClasspath.from configurations.annotationProcessor
107:     compileClasspath.from configurations.compileClasspath
108:     destinationDirectory = layout.buildDirectory.dir('my-classes')
109:     resources.from fileTree('src/main/resources')
110:     finalizedBy buildWar
111: }
112:
113: tasks.register('uwuifySources', UwuifySourcesTask) {
114:     sourceDirectory = layout.projectDirectory.dir('src/main/java')
115:     outputDirectory = layout.buildDirectory.dir('uwu-src')
116:     finalizedBy alt
117: }
118:
119: tasks.register('myBuild') {
120:     dependsOn 'myCompile'
121:     finalizedBy 'buildWar'
122: }
123:
124:
125: tasks.named('buildWar') {
126:     doFirst {
127:         def requested = gradle.startParameter.taskNames

```

```

128:         if (requested.contains('buildWar')) {
129:             throw new GradleException(
130:                 "Do not invoke 'buildWar' directly. Use 'myBuild'
or 'alt' instead."
131:             )
132:         }
133:     }
134: }
135:
136: test {
137:     useJUnitPlatform()
138:     jvmArgs "-javaagent:${configurations.testRuntimeClasspath.find
{ it.name.contains('mockito-core') }}"
139: }
140:
141: // Task types
142: import com.github.javaparser.ParserConfiguration
143: import com.github.javaparser.JavaParser
144: import com.github.javaparser.ast.CompilationUnit
145: import com.github.javaparser.ast.body.ClassOrInterfaceDeclaration
146: import java.util.regex.Pattern
147:
148: abstract class MyJavaCompile extends DefaultTask {
149:     @InputFiles
150:     abstract ConfigurableFileCollection getSource()
151:     @InputFiles
152:     abstract ConfigurableFileCollection getCompileClasspath()
153:     @InputFiles
154:     abstract ConfigurableFileCollection getProcessorClasspath()
155:     @InputFiles
156:     abstract ConfigurableFileCollection getResources()
157:
158:     @OutputDirectory
159:     abstract DirectoryProperty getDestinationDirectory()
160:
161:     @Inject
162:     abstract ExecOperations getExecOperations()
163:     @Inject
164:     abstract FileSystemOperations getFileSystemOperations()
165:
166:     @TaskAction
167:     void compile() {
168:         def destDir = destinationDirectory.get().asFile
169:         destDir.mkdirs()
170:
171:         File generatedSourcesDir = new File(getTemporaryDir(),
'generated-sources')
172:         generatedSourcesDir.mkdirs()
173:
174:         def sourceFiles = source.filter { it.name.endsWith('.java') }
*.absolutePath
175:         def cp = compileClasspath.asPath
176:         def processorPath = processorClasspath.asPath

```

```

177:
178:         execOperations.exec {
179:             commandLine = ['javac',
180:                 '-cp', cp,
181:                 '-processorpath', processorPath,
182:                 '-s', generatedSourcesDir.absolutePath,
183:                 '-d', destDir.absolutePath,
184:                 *sourceFiles
185:             ]
186:         }
187:         fileSystemOperations.copy {
188:             from resources
189:             into destinationDirectory
190:         }
191:     }
192: }
193:
194: abstract class MyBuildWar extends DefaultTask {
195:     @Input
196:     abstract Property<String> getBaseName()
197:     @InputDirectory
198:     abstract DirectoryProperty getSourceDirectory()
199:     @InputFiles
200:     abstract ConfigurableFileCollection getRuntimeClasspath()
201:     @Inject
202:     abstract ExecOperations getExecOperations()
203:     @OutputFile
204:     abstract RegularFileProperty getOutputFile()
205:
206:     @TaskAction
207:     void build() {
208:         File stagingDir = getTemporaryDir()
209:         File classesDir = new File(stagingDir, 'WEB-INF/classes')
210:         File libDir = new File(stagingDir, 'WEB-INF/lib')
211:         classesDir.mkdirs()
212:         libDir.mkdirs()
213:
214:         File srcDir = sourceDirectory.get().asFile
215:         srcDir.eachFileRecurse { File f ->
216:             if (f.isFile()) {
217:                 String rel =
srcDir.toURI().relativize(f.toURI()).toString()
218:                 File dest = new File(classesDir, rel)
219:                 dest.parentFile.mkdirs()
220:                 dest.bytes = f.bytes
221:             }
222:         }
223:         runtimeClasspath.each { File jar ->
224:             new File(libDir, jar.name).bytes = jar.bytes
225:         }
226:
227:         String outputPath = outputFile.get().asFile.absolutePath
228:

```

```

229:         String stagingPath = stagingDir.absolutePath
230:
231:         execOperations.exec {
232:             commandLine = ['jar', '-cf', outputPath, '-C',
stagingPath, '.']
233:         }
234:     }
235: }
236:
237: abstract class UwuiifySourcesTask extends DefaultTask {
238:     @InputDirectory
239:     abstract DirectoryProperty getSourceDirectory()
240:     @OutputDirectory
241:     abstract DirectoryProperty getOutputDirectory()
242:
243:     static String uwuifyString(String word) {
244:         word.replaceAll('r', 'w')
245:             .replaceAll('l', 'w')
246:             .replaceAll('R', 'W')
247:             .replaceAll('L', 'W')
248:     }
249:
250:     @TaskAction
251:     void execute() {
252:         File sourceDir = sourceDirectory.get().asFile
253:         File outputDir = outputDirectory.get().asFile
254:         outputDir.mkdirs()
255:
256:         List<File> javaFiles = []
257:         sourceDir.eachFileRecurse { File f ->
258:             if (f.isFile() && f.name.endsWith('.java')) {
259:                 javaFiles.add(f)
260:             }
261:         }
262:
263:         Map<String, String> classNameMappings = [:]
264:         Map<String, Map<String, String>> fileFieldMappings = [:]
265:         Map<String, String> fileRenames = [:]
266:
267:         javaFiles.each { File sourceFile ->
268:             ParserConfiguration config = new ParserConfiguration()
269:             config.setLanguageLevel(ParserConfiguration.LanguageLevel.JAVA_17)
270:             def parser = new JavaParser(config)
271:             CompilationUnit cu = parser.parse(sourceFile).result.get()
272:             Map<String, String> fieldMappings = [:]
273:
274:             cu.findAll(ClassOrInterfaceDeclaration.class).each
275:             { ClassOrInterfaceDeclaration clazz ->
276:                 String oldClassName = clazz.nameAsString
277:                 String newClassName = uwuifyString(oldClassName)
278:                 classNameMappings[oldClassName] = newClassName
279:
280:                 String newFileName = newClassName + '.java'

```



```

280:         if (sourceFile.name != newFileName) {
281:             fileRenames[sourceFile.absolutePath] = newFileName
282:         }
283:
284:         boolean hasLombokAnnotation = clazz.annotations.any
285:         { ann ->
286:             String annName = ann.nameAsString
287:             annName in ['Data', 'Getter', 'Setter']
288:         }
289:         if (hasLombokAnnotation) {
290:             clazz.fields.each { field ->
291:                 field.variables.each { var ->
292:                     String oldFieldName = var.nameAsString
293:                     String newFieldName =
294:                         uwuifyString(oldFieldName)
295:                     fieldMappings[oldFieldName] = newFieldName
296:                 }
297:             }
298:         }
299:         fileFieldMappings[sourceFile.absolutePath] = fieldMappings
300:     }
301:
302:     javaFiles.each { File sourceFile ->
303:         String content = sourceFile.text
304:         Map<String, String> currentFileFieldMappings =
305:             fileFieldMappings[sourceFile.absolutePath] ?: [:]
306:
307:         classNameMappings.each { oldName, newName ->
308:             if (oldName != newName) {
309:                 content = content.replaceAll('class\\s+' +
310:                     Pattern.quote(oldName) + '\\b', 'class ' + newName)
311:                 content = content.replaceAll('interface\\s+' +
312:                     Pattern.quote(oldName) + '\\b', 'interface ' + newName)
313:                 content = content.replaceAll('public\\s+' +
314:                     Pattern.quote(oldName) + '\\s*\\(', 'public ' + newName + '(')
315:                 content = content.replaceAll('protected\\s+' +
316:                     Pattern.quote(oldName) + '\\s*\\(', 'protected ' + newName + '(')
317:                 content = content.replaceAll('private\\s+' +
318:                     Pattern.quote(oldName) + '\\s*\\(', 'private ' + newName + '(')
319:                 content = content.replaceAll('new\\s+' +
320:                     Pattern.quote(oldName) + '\\s*\\(', 'new ' + newName + '(')
321:                 content = content.replaceAll('new\\s+' +
322:                     Pattern.quote(oldName) + '<', 'new ' + newName + '<')
323:                 content = content.replaceAll('\\s' +
324:                     Pattern.quote(oldName) + '\\s', ' ' + newName + ' ')

```

```

320:         content = content.replaceAll('\s' +
Pattern.quote(oldName) + '<', ' ' + newName + '<')
321:         content = content.replaceAll('\(' +
Pattern.quote(oldName) + '\\b', '(' + newName)
322:         content = content.replaceAll(',\s*' +
Pattern.quote(oldName) + '\\b', ', ' + newName)
323:         content = content.replaceAll('\s' +
Pattern.quote(oldName) + ', ', ' ' + newName + ',')
324:         content = content.replaceAll('<' +
Pattern.quote(oldName) + '>', '<' + newName + '>')
325:
326:         content = content.replaceAll('\\b' +
Pattern.quote(oldName) + '\\.class\\b', newName + '.class')
327:
328:         content = content.replaceAll('\\.' +
Pattern.quote(oldName) + ';', '.' + newName + ';')
329:
330:         content = content.replaceAll('throws\\s+' +
Pattern.quote(oldName) + '\\b', 'throws ' + newName)
331:     }
332: }
333:
334: fileFieldMappings.values().each { fileFieldMap ->
335:     fileFieldMap.each { oldFieldName, newFieldName ->
336:         if (oldFieldName != newFieldName) {
337:             String getterName = 'get' +
oldFieldName.substring(0,1).toUpperCase() + oldFieldName.substring(1)
338:             String newGetterName = 'get' +
newFieldName.substring(0,1).toUpperCase() + newFieldName.substring(1)
339:             content = content.replaceAll('\\.' +
Pattern.quote(getterName) + '\\s*\\(', '.' + newGetterName + '(')
340:
341:             String setterName = 'set' +
oldFieldName.substring(0,1).toUpperCase() + oldFieldName.substring(1)
342:             String newSetterName = 'set' +
newFieldName.substring(0,1).toUpperCase() + newFieldName.substring(1)
343:             content = content.replaceAll('\\.' +
Pattern.quote(setterName) + '\\s*\\(', '.' + newSetterName + '(')
344:         }
345:     }
346: }
347:
348: currentFileFieldMappings.each { oldFieldName, newFieldName
->
349:     if (oldFieldName != newFieldName) {
350:         content = content.replaceAll('this\\s*\\.\\s*' +
Pattern.quote(oldFieldName) + '\\b', 'this.' + newFieldName)
351:
352:         content = content.replaceAll('super\\s*\\.\\s*' +
Pattern.quote(oldFieldName) + '\\b', 'super.' + newFieldName)
353:

```

```

354:             content = content.replaceAll('\\b' +
Pattern.quote(oldFieldName) + '(?=\s*([=;]))', newFieldName)
355:         }
356:     }
357:
358:     String relPath =
sourceDir.toURI().relativize(sourceFile.toURI()).toString()
359:     String baseName = new File(relPath).name
360:     String dirPath = relPath.substring(0, relPath.length() -
baseName.length())
361:
362:     String newFileName =
fileRenames[sourceFile.absolutePath] ?: sourceFile.name
363:
364:     File outputFile = new File(outputDir, dirPath +
newFileName)
365:     outputFile.parentFile.mkdirs()
366:     outputFile.text = content
367: }
368: }
369: }
370:

```

3. Пример исполнения цели alt

```

1: package ru.astrosoup.weblab3.DT0s.authorisation;
2:
3: import lombok.Data;
4:
5: @Data
6: public class WoginDto {
7:     private String usewname;
8:     private String passowd;
9: }
10:

```

4. Листинг тестов

```

1: package ru.astrosoup.weblab3.services.hit;
2:
3: import org.junit.jupiter.api.BeforeEach;
4: import org.junit.jupiter.api.Test;
5: import org.junit.jupiter.api.extension.ExtendWith;
6: import org.mockito.InjectMocks;
7: import org.mockito.Mock;
8: import org.mockito.junit.jupiter.MockitoExtension;
9: import ru.astrosoup.weblab3.DA0s.authorisation.UserRepository;
10: import ru.astrosoup.weblab3.DA0s.hit.HitRepository;
11: import ru.astrosoup.weblab3.DT0s.authorisation.JwtDto;
12: import ru.astrosoup.weblab3.DT0s.hit.AreaHitDto;
13: import ru.astrosoup.weblab3.DT0s.hit.AreaHitRequest;
14: import ru.astrosoup.weblab3.DT0s.hit.AreaHitResponse;
15: import ru.astrosoup.weblab3.entities.authorisation.UserEntity;
16: import ru.astrosoup.weblab3.entities.hit.HitEntity;
17: import ru.astrosoup.weblab3.exceptions.InvalidHitRequestException;

```

```

18: import ru.astrosoup.weblab3.exceptions.UserDoesNotExistException;
19: import ru.astrosoup.weblab3.services.hit.HitService;
20:
21: import java.time.LocalDate;
22: import java.util.Optional;
23:
24: import static org.junit.jupiter.api.Assertions.assertEquals;
25: import static org.junit.jupiter.api.Assertions.assertThrows;
26: import static org.mockito.AdditionalAnswers.returnsFirstArg;
27: import static org.mockito.ArgumentMatchers.any;
28: import static org.mockito.Mockito.lenient;
29: import static org.mockito.Mockito.when;
30:
31: @ExtendWith(MockitoExtension.class)
32: public class HitServiceTest {
33:
34:     @Mock
35:     UserRepository userRepository;
36:
37:     @Mock
38:     HitRepository hitRepository;
39:
40:     @InjectMocks
41:     HitService hitService;
42:
43:     @BeforeEach
44:     public void setupMocks() {
45:         lenient().when(userRepository.findById(1L)).thenReturn(Optional.of(new
46:             UserEntity(1L, "Serge Klimenkov", "TotallyPasswordHash")));
47:         lenient().when(hitRepository.save(any(HitEntity.class))).thenReturn(returnsFir
48:             );
49:
50:         @Test
51:         public void testThatGoodHitIsRegisteredCorrectly() {
52:             assertEquals(hitService.addHit(hitDto(goodHitRequest())),
53:                 goodHitResponse());
54:         }
55:
56:         @Test
57:         public void testThatBadHitIsRegisteredCorrectly() {
58:             assertEquals(hitService.addHit(hitDto(badHitRequest())),
59:                 badHitResponse());
60:         }
61:
62:         @Test
63:         public void testThatInvalidHitThrows() {
64:             assertThrows(InvalidHitRequestException.class, () ->
65:                 hitService.addHit(hitDto(InvalidHitRequest())));
66:         }
67:
68:         @Test
69:         public void testThatInvalidUserThrows() {
70:             when(userRepository.findById(1L)).thenReturn(Optional.empty());
71:         }
72:     }

```

```

66:         assertThrows(UserDoesNotExistException.class, () ->
hitService.addHit(hitDto(goodHitRequest())));
67:     }
68:
69:     private AreaHitDto hitDto(AreaHitRequest request) {
70:         return new AreaHitDto(new JwtDto(1L, null), request);
71:     }
72:
73:
74:     private AreaHitRequest goodHitRequest() {
75:         return new AreaHitRequest(3, 1, 1.0f);
76:     }
77:     private AreaHitResponse goodHitResponse() {
78:         return new AreaHitResponse(3, 1, 1.0f, true, LocalDate.now());
79:     }
80:
81:     private AreaHitRequest badHitRequest () {
82:         return new AreaHitRequest(3, 3, 5f);
83:     }
84:     private AreaHitResponse badHitResponse() {
85:         return new AreaHitResponse(3, 3, 5f, false, LocalDate.now());
86:     }
87:
88:     private AreaHitRequest InvalidHitRequest() {
89:         return new AreaHitRequest(999, 999, 999f);
90:     }
91: }
92:

```

5. Вывод

В ходе лабораторной работы я преумножил свои знания в написании билд скриптов для системы сборки gradle, написал модульные тесты на JUnit 5 с использованием Mockito, разобрался с внутренней работой gradle.

